

COMPUTER NETWORKS

Week-13

CRC

- CRC actually performs arithmetic using:
- Polynomial division
- Modulo-2 arithmetic
- XOR instead of subtraction
- That is why CRC is also called:
- Polynomial code error detection

Modulo-2 arithmetic

- It works with only two digits: 0 and 1
- The important rule is: There are no carries and no borrows

CRC and Modulo-2 arithmetic

- CRC performs Polynomial division using XOR instead of subtraction
- In CRC:
- Addition = XOR
- Subtraction = XOR
- Modulo-2 arithmetic is simply XOR arithmetic

CRC- Example

- Polynomial form of binary data at the sender end
- $x^7 + x^6 + x^3 + 1$
- $1.x^7 + 1.x^6 + 0 + 0 + 1.x^3 + 0 + 0 + 1.x^0$
- Data in binary?

CRC- Example

- Polynomial form of binary data at the sender end
- $x^7 + x^6 + x^3 + 1$
- Data in bits : 11001001

CRC- Example

- Polynomial form of binary data at the sender end
- $x^7 + x^6 + x^3 + 1$
- Data in bits : 11001001
- Generator Polynomial : $x^3 + 1$
- Generator in binary : 1001

More on Parity Check

- Two-dimensional Parity Check

Two-dimensional Parity Check

- An error-detection technique used in data communication
- It extends the idea of simple parity (even/odd bits) into both rows and columns, making it more powerful for detecting errors.

Two-dimensional Parity Check

- **Arrange data in a grid (matrix)**
- Block of bits arranged in rows and columns
- **Row parity (horizontal check)**
- For each row, add an extra bit (parity bit) to ensure even (or odd) parity
- **Column parity (vertical check)**
- After adding row parity bits, compute parity for each column and add an extra row of parity bits

Two-dimensional Parity Check

- Original Data

1	0	1	1
0	1	0	1
1	1	0	0
0	0	1	1

Two-dimensional Parity Check

- Using Even Parity
- Add the row parity

				Row Parity
1	0	1	1	1
0	1	0	1	0
1	1	0	0	0
0	0	1	1	0

Two-dimensional Parity Check

- Using Even Parity
- Add Column Parity

					Row Parity
	1	0	1	1	1
	0	1	0	1	0
	1	1	0	0	0
	0	0	1	1	0
Col Parity	0	0	0	1	

Two-dimensional Parity Check

- Using Even Parity
- Add Column Parity

					Row Parity
	1	0	1	1	1
	0	1	0	1	0
	1	1	0	0	0
	0	0	1	1	0
Col Parity	0	0	0	1	1

2-D Parity Check - Error Detection

- If a **single-bit** error occurs:
 - One row parity and one column parity will fail
 - Their intersection pinpoints the exact erroneous bit
 - Can be corrected
- If **multiple** errors occur:
 - Some may be detected, but not always corrected
 - Certain patterns (e.g., errors in a rectangle) may go undetected

Ex-1

- Original Data

1	0	1	1
0	1	0	1
1	1	1	0
0	0	1	0

Ex-1

- Using Even Parity

				Row Parity
1	0	1	1	1
0	1	0	1	0
1	1	1	0	1
0	0	1	0	1

Ex-1

- Using Even Parity

					Row Parity
Col Parity	1	0	1	1	1
	0	1	0	1	0
	1	1	1	0	1
	0	0	1	0	1
	0	0	1	0	1

Data Transmitted

1	0	1	1	1	0	1	0	1	0	1	1	1	0	1	0	0	1	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Ex-1

- Suppose one bit changes accidentally
- Third row, second column changes from 1 → 0
- Received data becomes:

					Row Parity
	1	0	1	1	1
	0	1	0	1	0
	1	0	1	0	1
	0	0	1	0	1
Col Parity	0	0	1	0	1

Ex-1

- The Receiver Checks parity
- Row 3 shows an error
- Column 2 shows an error
- Their intersection identifies the corrupted bit
- The receiver know exactly which bit is wrong
- The incorrect bit is 0
- It is corrected back to 1

Adv

- Detect all single-bit errors
- Identify the exact wrong bit
- Correct single-bit errors automatically
- Stronger than simple one-dimensional parity checking.
- Simple to implement
- **Limitations**
- Cannot reliably detect all multiple-bit errors
- Adds extra redundancy (overhead)

Two-dimensional Parity Check

- Used in :
- Serial communication systems
- Wireless sensor devices
- Low-cost embedded systems
- RAM modules
- Cache memory
- RAID systems use parity infor across disks. If one disk fails, parity hlps recover data
- Servers and banks also us an advanced version of this technique

Modern Networks

- Use stronger techniques like:
- CRC (Cyclic Redundancy Check)
- Checksum
- Because they detect burst errors much better than parity methods

Visual Representation

- **Single bit error**



In a frame, there is only one bit, anywhere though, which is corrupt.

- **Multiple bits error**



Frame is received with more than one bits in corrupted state.

- **Burst error**



Frame contains more than 1 consecutive bits corrupted.

Msg. Authentication

- Data Integrity
- Source Authentication

Data Integrity

- To ensure that data arriving at the destination is in its original form

Source Authentication

- Data arriving is coming from an authentic source

Message Authentication

- No modification
- No insertion
- No deletion
- No replay

Replay Attack

- In the story *Ali Baba and the Forty Thieves*, the robber captain utters “Open Sesame!” to gain entrance to the cave containing the thieves’ stolen goods. After overhearing the phrase, Ali Baba reuses it to open the doorway to the cave himself, thus engaging in a replay attack.
- In computing, a replay attack occurs when transmitted authentication or access control information is intercepted and then re-transmitted to either produce an unauthorized effect or gain unauthorized access
- <https://www.comparitech.com/blog/information-security/what-is-a-replay-attack/>

Cryptographic Hash Functions Applications

- Message authentication
- Digital signature
- Other Applications (Internet protocols)

Hash Function

- A hash function maps a message of an arbitrary length to a m-bit output
- The output is known as the **fingerprint** or the **message digest**
- Even a tiny change in the input produces a completely different hash
- Same input always gives the same hash.
- Examples of popular hash functions are SHA-256 (output length is 256 bit hash), MD5 (128 bit long hash), Bcrypt, RIPEMD

Hash functions

- Instead of using a key, we use hash function
- A hash function (H) accepts a variable length block (eg : your email password, the full text of the email, or even some video) of data (M) as input and produces a fixed size hash value (h)
- Hash value is also called msg. digest
- $h = H(M)$
- h acts as an authenticator
- Cannot get msg (M) back from the hash value (one-way)

One way Hash Functions

- You have a hash function (H)
- It takes a message (M)
- Produces hash value (h)
- Given an input, it is **easy to compute** the output. But it is practically impossible to reverse engineer a hash output and derive the input
- Hence a hash function is also called a **one-way function**
- You cannot use hashing for encryption and decryption (as decryption is impossible due to the one-way nature)

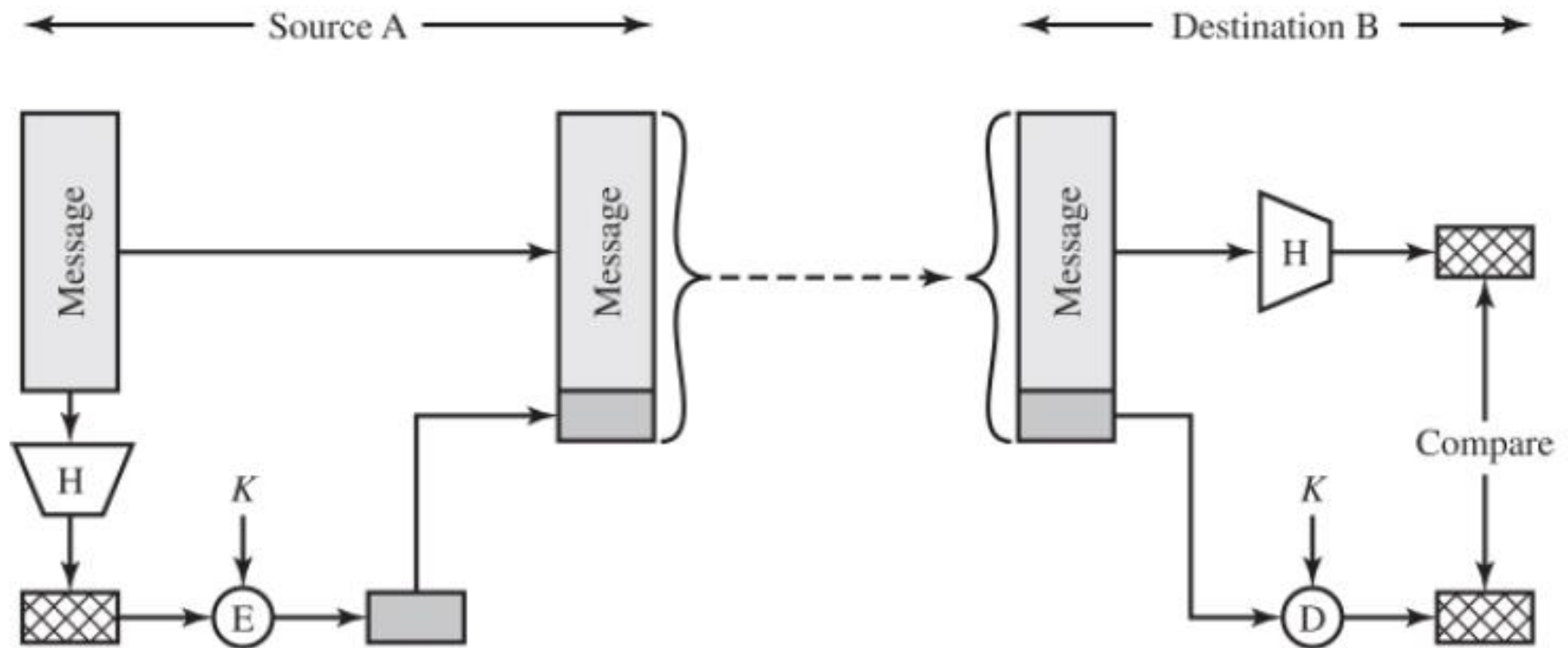
Hash output

- A hash output is useful to represent an input
- It is called a **fingerprint**
- This is useful if you want to make sure your data is not tampered or corrupted when it travels in a network
- The hash of “sent data” should always equal the hash of “received data”
- Comparison of data is the most common use of hashing

1. Using Symmetric Encryption

- The message digest can be encrypted using symmetric encryption key
- Authenticity is assured by assuming that only the sender and receiver share the encryption key

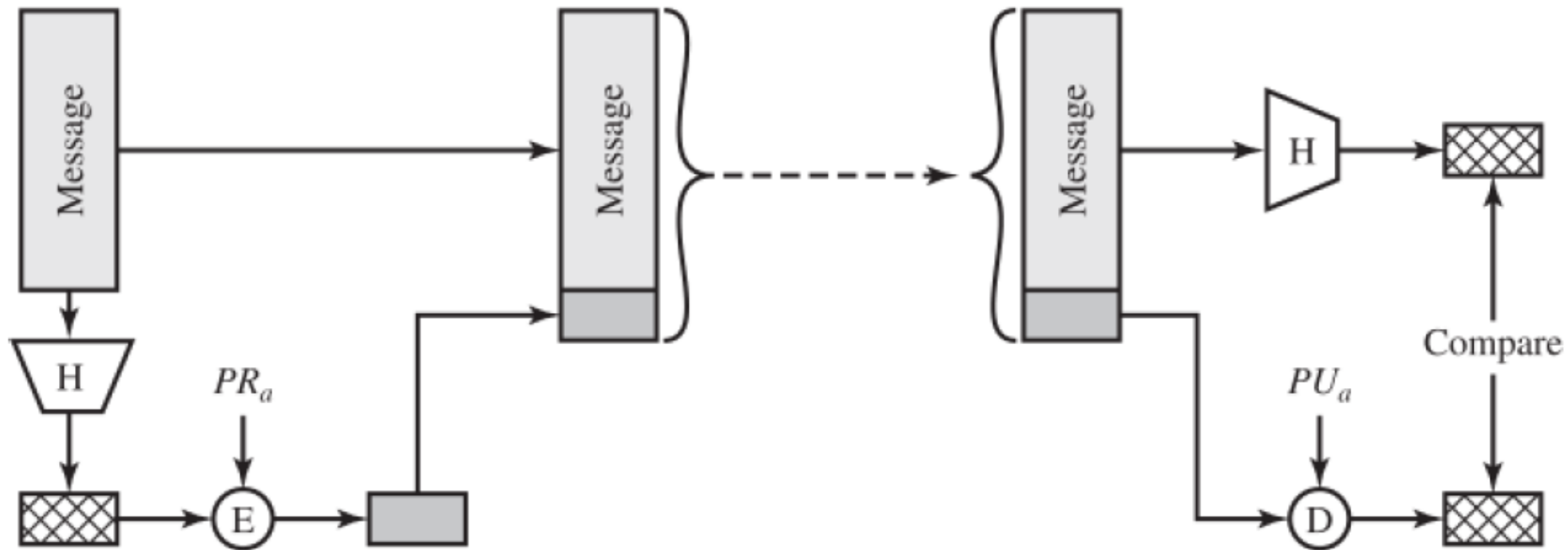
1. Using Symmetric Encryption



2. Using public key encryption

- The message digest can also be encrypted using public-key encryption
- Provides message authentication
- Does not require the distribution of keys to communicating parties

2. Using public key encryption



Digital Signatures: The Problem

- Consider the real-life example where a person pays by credit card and signs a bill; the seller verifies that the signature on the bill is the same with the signature on the card
- Contracts, they are valid if they are signed.
- Signatures provide non-repudiation
- Can we have a similar service in the electronic world?
 - Does Message Authentication Code provide non-repudiation? Why?

Non-repudiation

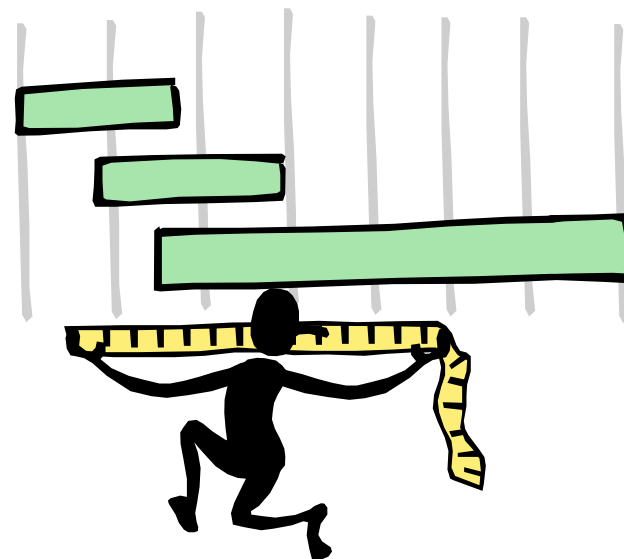
- A common example of non-repudiation in cybersecurity occurs in e-commerce
- When a customer makes an online purchase and signs off using their private key (via digital signature), they cannot later claim that they did not authorize the transaction.

Digital Signatures

- MAC: One party generates MAC, one party verifies integrity.
- Digital signatures: One party generates signature, many parties can verify
- Digital Signature: a data string which associates a message with some originating entity
- Digital Signature Scheme:
 - a signing algorithm: takes a message and a (private) signing key, outputs a signature
 - a verification algorithm: takes a (public) key verification key, a message, and a signature
- Provides:
 - Authentication, Data integrity, Non-Repudiation

Digital Signatures and Hash

- Very often digital signatures are used with hash functions, hash of a message is signed, instead of the message.
- Hash function must be:
 - Pre-image resistant
 - Weak collision resistant
 - Strong collision resistant



Book-Reference

- Chapter 29, Douglas E. Comer
- Chp 6 James W Kurose